

Library Management System

Prepared by Students of
Software Engineering Department

Name	ID
Mai Ashraf	192300095
Malak Mohamed	192300094
Gana Ahmed	192300067
Maram Moustafa	192300280
Menna allah mostafa	192300166

Supervised by: Dr. Sabah saad

TA. Fady roumany

1. Introduction	2
2. Project Overview.....	3
3. System Architecture	4
3.1 Frontend Layer	4
3.2 Backend Layer.....	5
3.3 API Layer	5
4. Testing Methodology	6
4.1 Manual Testing	6
4.2 API Testing	6
4.3 Automation Testing.....	6
4.4 Functional Testing	7
4.5 Black Box Testing.....	7
5. Manual Testing	7
7. Automation Testing using Selenium	9
8. Test Execution Summary	10
8.1 Overall Testing Status	11
Testing Type.....	11
Status	11
8.2 API Testing Summary	11
8.3 Selenium Automation Summary	11
9. Bug Report	12
Bug 1: Duplicate Books Allowed	12
Bug 2: Empty Search Field Behavior.....	13
Bug 3: Backend Design Weakness (No Unique Identifier).....	13
Bug 4: Global Authentication Variable.....	13
10. Tools Used in Development and Testing.....	14

11. Challenges Faced	14
12. Lessons Learned.....	15
13. Conclusion.....	17

1. Introduction

Software testing is considered one of the most essential phases in the Software Development Life Cycle (SDLC), as it ensures that software applications operate correctly according to the specified requirements. Testing plays a significant role in identifying defects, improving software quality, enhancing system performance, and ensuring a reliable and user-friendly experience.

This project demonstrates a comprehensive testing process for a web-based Library Management System. The system provides several functionalities, including user login, adding books, displaying available books, searching for books, borrowing and returning books, deleting books, and viewing library statistics.

To ensure the reliability and quality of the system, multiple testing approaches were applied throughout the project, including:

- Manual Testing
- Functional Testing
- UI Testing
- Backend Testing
- API Testing
- Automation Testing

The testing activities were carried out using various professional tools and technologies such as Selenium WebDriver for automation testing, Postman for API testing, Excel for managing test cases and reports, IntelliJ IDEA as the development environment, and GitHub Pages for deploying the website.

The primary objective of this project is to validate the functionality of all system components, detect possible defects and weaknesses, and ensure that the application performs efficiently and accurately under different testing scenarios.

Additionally, the project aims to demonstrate the importance of combining manual and automated testing techniques to achieve higher software quality and reliability.

2. Project Overview

The Library Management System is a web-based application designed to simplify and manage library operations in a digital environment. The system provides users with an easy and interactive interface for managing books and performing different library-related tasks efficiently.

The application includes several core functionalities that help simulate the operations of a real library management system. These functionalities include:

- User Authentication (Login)
- Adding New Books
- Viewing Available Books
- Searching for Books
- Borrowing Books
- Returning Borrowed Books
- Deleting Books
- Viewing Library Statistics
- Displaying Borrowed Books

The project was developed using modern web technologies to ensure simplicity, responsiveness, and ease of use.

➤ Frontend Technologies

- **HTML**
- **CSS**
- **JavaScript**

➤ Backend Technologies

- Node.js
- Express.js

Deployment

The website was deployed using GitHub Pages to make the application accessible online and easy to test from different environments.

Project URL:

[Library Management System Website](#)

3. System Architecture

The Library Management System follows a multi-layer architecture design to separate the presentation layer, business logic layer, and communication layer. This architecture improves system organization, maintainability, scalability, and testing efficiency.

The system architecture consists of three main layers:

3.1 Frontend Layer

The frontend layer is responsible for handling user interaction and displaying the graphical user interface (GUI). It was developed using modern web technologies including HTML, CSS, and JavaScript to provide an interactive and responsive user experience.

Main Frontend Features

- Dynamic tab navigation
- Responsive user interface
- Toast notifications

- **Form validation**
- **Interactive book management**

3.2 Backend Layer

The backend layer was developed using Node.js and Express.js. It is responsible for processing requests, implementing business logic, and managing communication between the frontend and the data layer through RESTful APIs.

Backend Responsibilities

- **Authentication handling**
- **Data processing**
- **Borrow and return operations**
- **Book management operations**
- **Error handling and validation**

3.3 API Layer

The API layer enables communication between the frontend and backend components using HTTP requests and RESTful services.

Implemented API Endpoints

Method	Endpoint	Function
POST	/login	User login
POST	/books	Add new book
GET	/books	Retrieve all books
PUT	/borrow	Borrow a book
PUT	/return	Return a borrowed book

DELETE

/books/:title

Delete a book

4. Testing Methodology

Several testing methodologies were applied throughout the project to ensure the quality, reliability, and functionality of the Library Management System. Different testing approaches were used to validate both frontend and backend components of the application.

4.1 Manual Testing

Manual testing was conducted using structured Excel test cases to validate the overall system functionality and user experience.

Manual Testing Validation Areas

- **Functional requirements**
- **User interface behavior**
- **Input validation**
- **Error handling**

4.2 API Testing

API testing was performed using Postman to validate the communication between the frontend and backend services.

API Testing Validation Areas

- **HTTP requests and responses**
- **API functionality**
- **Response messages**
- **Error codes and validation**

- **Authentication behavior**

4.3 Automation Testing

Automation testing was implemented using Selenium WebDriver with Java to automate repetitive testing tasks and simulate real user interactions with the system.

Automated Test Scenarios

- **User login**
- **Adding books**
- **Viewing books**
- **Searching for books**
- **Borrowing books**
- **Returning books**
- **Deleting books**

4.4 Functional Testing

Functional testing was applied to ensure that each feature of the system behaves according to the specified functional requirements and produces the expected outputs.

4.5 Black Box Testing

Black box testing techniques were used to evaluate the system functionality without analyzing the internal source code or implementation details. The testing process focused on validating inputs, outputs, and system behavior from the user perspective.

5. Manual Testing

Manual testing was conducted using organized Excel test cases covering the major functionalities of the Library Management System.

Tested Modules

- **Login**
- **Add Book**
- **View Books**
- **Search Book**
- **Borrow Book**
- **Return Book**
- **Delete Book**
- **Statistics**

Manual Testing Objectives

- Verify system behavior
- Validate user inputs
- Detect UI issues
- Validate business logic
- Ensure correct navigation

The manual testing process helped identify functional issues, validate application behavior, and improve the overall user experience and system reliability.

6. API Testing using Postman

API testing was performed using Postman to validate backend functionality.

Total API Test Cases

- 15+ API test cases

API Testing Scenarios

- **Authentication Testing**
 - Valid login
 - Invalid username
 - Invalid password
 - Empty login fields
- **Book Management Testing**
 - Add valid book
 - Add invalid book
 - View books
 - Delete books
- **Borrowing System Testing**
 - Borrow available book
 - Borrow unavailable book
 - Return borrowed book
 - Return non-borrowed book

API Test Results

Result	Count
Passed	15
Failed	0
Success Rate	100%

The API layer demonstrated stable and reliable behavior during all executed test scenarios.

7. Automation Testing using Selenium

Automation testing was implemented using Selenium WebDriver with Java and Maven to simulate real user interactions and validate the end-to-end functionality of the Library Management System.

➤ **Technologies Used**

- **Selenium WebDriver 4.20.0**
- **Java 17**
- **Maven**
- **WebDriverManager**
- **IntelliJ IDEA**

➤ **Automated Test Flow**

The automation script simulates a complete end-to-end user workflow, which includes the following steps:

- **Launch the web application**
- **Perform user login with valid credentials**
- **Add new books to the system**
- **View available books**
- **Search for specific books**
- **Borrow books**
- **View system statistics**
- **Delete books**
- **Verify successful deletion**
- **Close the browser session**

➤ **Advantages of Automation Testing**

- **Faster execution compared to manual testing**
- **Reduction of human errors**
- **Reusability of test scripts**
- **Efficient regression testing**
- **Improved overall testing efficiency**

➤ **Limitations of Automation Testing**

- **Use of static waits (`Thread.sleep`), which may reduce efficiency**
- **Lack of assertions for result validation**

- No integration with testing frameworks such as **TestNG** or **JUnit**

Despite these limitations, the automation testing successfully simulated realistic end-to-end user workflows and contributed significantly to validating the system's core functionality.

8. Test Execution Summary

This section provides an overall summary of the testing execution results across all testing types applied in the Library Management System project. The results demonstrate the effectiveness and stability of the system under different testing scenarios.

8.1 Overall Testing Status

Testing Type	Status
Manual Testing	Completed
API Testing	Completed
Automation Testing	Completed
Functional Testing	Completed

8.2 API Testing Summary

The API testing phase was executed using Postman to validate system endpoints and ensure correct response behavior.

Metric	Result
Total API Tests	15
Passed Tests	15
Failed Tests	0
Success Rate	100%

The results indicate that all API endpoints function correctly and return expected responses without errors.

8.3 Selenium Automation Summary

The automation testing was executed using Selenium WebDriver to validate key user workflows.

Scenario	Status
Login	Passed
Add Book	Passed
View Books	Passed
Search Book	Passed
Borrow Book	Passed
Delete Book	Passed
Statistics	Passed

The results confirm that all automated test scenarios executed successfully and the system behaves as expected during end-to-end workflows.

9. Bug Report

During the testing process, several issues were identified in the Library Management System. The following section describes the discovered bugs, their reproduction steps, expected behavior, actual behavior, and severity level.

Bug 1: Duplicate Books Allowed

➤ **Description:**

The system allows users to add multiple books with the same title and author name without any validation.

➤ **Steps to Reproduce:**

- **Login to the system**
- **Add a new book**
- **Add another book with the same title and author**

➤ **Expected Result:**

The system should prevent duplicate book entries and display a validation message.

➤ **Actual Result:**

Duplicate books are successfully added to the system.

- **Severity:** Medium

Bug 2: Empty Search Field Behavior

➤ **Description:**

When the search input field is left empty and the search button is clicked, the system displays an incorrect message.

➤ **Steps to Reproduce:**

- **Open the Search tab**
- **Leave the input field empty**
- **Click the Search button**

➤ **Expected Result:**

The system should display a validation message such as:
“Please enter a book title.”

➤ **Actual Result:**

The system displays: “Book not found.”

- **Severity:** Low

Bug 3: Backend Design Weakness (No Unique Identifier)

➤ **Description:**

Books in the system are identified only by title without using a unique identifier (ID).

➤ **Impact:**

This may lead to conflicts when multiple books share the same title and makes data management less reliable.

- **Severity:** Medium

Bug 4: Global Authentication Variable

➤ **Description:**

The backend uses a global variable to manage user login state instead of session-based or token-based authentication.

➤ **Impact:**

This approach is not secure and not scalable for production-level applications.

- **Severity:** Medium

10. Tools Used in Development and Testing

This section presents the tools and technologies used throughout the development and testing phases of the Library Management System project. Each tool was selected to support a specific part of the software lifecycle, including development, automation, API testing, and deployment.

Tool	Purpose
Visual Studio Code	Frontend and backend development
IntelliJ IDEA	Selenium automation development

Selenium WebDriver	Automation testing framework
Postman	API testing and validation
Excel	Manual test case design and execution
GitHub Pages	Website deployment
Maven	Dependency management for Java project
WebDriverManager	Automatic browser driver handling

11. Challenges Faced

During the development and testing phases of the project, several technical and analytical challenges were encountered. These challenges contributed significantly to improving problem-solving, debugging, and testing skills.

➤ **The main challenges included:**

- **Synchronization issues while performing automation testing using Selenium WebDriver**
- **Managing communication and integration between frontend and backend components**
- **Debugging API requests and analyzing responses using Postman**
- **Handling different validation scenarios for user inputs**
- **Designing effective and comprehensive manual test cases**
- **Identifying logical weaknesses and design issues in the backend system**

Despite these challenges, overcoming them provided valuable hands-on experience in real-world software testing practices and improved overall understanding of system behavior and quality assurance processes.

12. Lessons Learned

This project provided valuable hands-on experience in the field of Software Testing and Quality Assurance. It helped in developing both technical and analytical skills through practical implementation of real-world testing scenarios.

Throughout the project, several key skills were acquired and strengthened:

➤ Technical Skills

- **API testing using Postman and validating RESTful services**
- **Automation testing using Selenium WebDriver**
- **Frontend testing of user interfaces and interactions**
- **Backend testing and validation of business logic**
- **Understanding and handling REST APIs**
- **Writing structured and detailed bug reports**

➤ Analytical and Problem-Solving Skills

- **Identifying functional and logical defects in the system**
- **Designing effective test cases for different scenarios**
- **Understanding system behavior under various inputs**
- **Debugging issues across frontend and backend layers**

➤ Communication and QA Skills

- **Documenting test cases and results in a structured format**
- **Reporting bugs with clear reproduction steps and severity levels**

- **Understanding the importance of software quality and reliability**

➤ **Overall Outcome**

The project significantly improved practical knowledge of software testing methodologies and enhanced the ability to analyze, test, and validate web applications in a structured and professional manner.

13. Conclusion

The Library Management System project successfully demonstrated a complete Software Testing Life Cycle through the application of manual testing, API testing, and automation testing techniques.

The system was thoroughly evaluated across both frontend and backend components, and the executed test cases confirmed that the application behaves as expected under various scenarios. This ensured that the core functionalities of the system are reliable and meet the required specifications.

In addition, the testing process helped in identifying several functional bugs and design weaknesses, which contributed to improving the overall quality, stability, and reliability of the system.

Throughout this project, valuable practical experience was gained in different areas of software engineering, including software testing, automation testing, API testing, functional testing, web application development, and quality assurance practices.

Overall, this project represents a practical implementation of modern software testing concepts and methodologies, and it reflects the importance of combining different testing approaches to ensure high-quality software systems.

